Python

Features of Python

- High Level
- Free and Open Source
- Case Sensitive
- Interpreted
- Plateform Independent
- Rich library of functions
- Dynamic
- General Purpose

Working with Python

To write and run python programs we need Python Interpreter also called Python IDLE.

Execution Mode

We can use Python Interpreter in two ways:

- Interactive mode
- Script mode

Interactive Mode

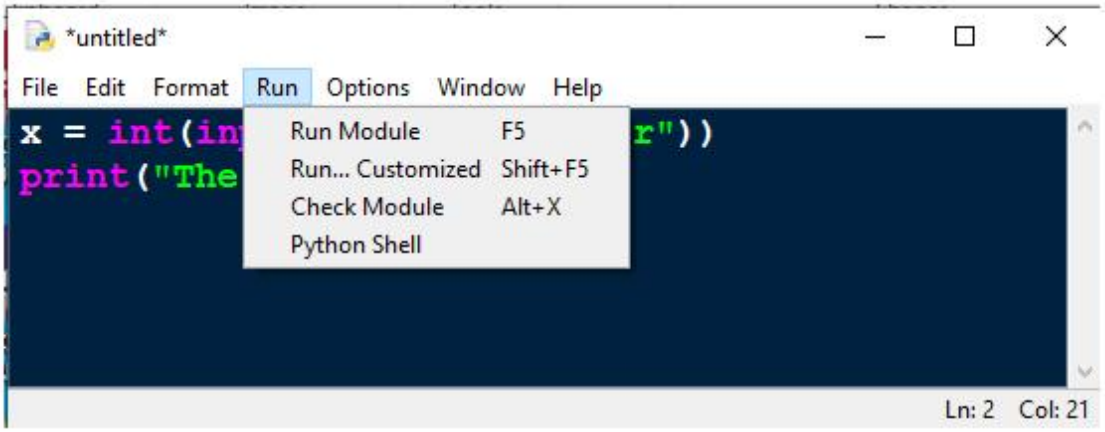
- Instant execution of individual statement
- Convenient for testing single line of code
- We cannot save statements for future use

Script Mode

- Allows us to write and execute more than one Instruction together.
- We can save programs (python script) for future use
- Python scripts are saved as file with extension ".py"

How to execute or run python program

- Open python IDLE
- Click 'File' and select 'New' to open Script mode
- Type source code and save it
- Click on 'Run' menu and select 'Run Module'



Python Keywords

- These are predefined words which a specific meaning to Python Interpreter.
- These are reserve keywords
- Keywords in python are case sensitive

False	class	finally	is	return	None	continue
for	lambda	try	True	def	from	nonlocal
while	and	del	global	not	with	as
elif	if	or	yield	assert	else	import
pass	break	except	in	raise		

www.tutorialspoint.in

Identifier

Identifiers are name used to identify a variable, function or any other entities in a programs.

Rules for naming Identifier

- The name should begin with an alphabet or and underscore sign and can be followed by any combination of charaters a-z, A-Z, 0-9 or underscore.
- It can be of any length but we should keep it simple, short and meaningful.
- it should not be a python keyword or reserved word.
- We cannot use special symbols like !, @, #, $, % etc. in identifiers.

Variables

- It can be referred as an object or element that occupies memory space which can contain a value.
- Value of variable can be numeric, alphanumeric or combination of both.
- In python assignment statement is used to create variable and assign values to it.

Data types

These are keywords which determine the type of data stored in a variable. Following table show data types used in python:

Data Types in Python			
Type	**Data Type**	**Description**	**Example**
Number	Int	Integer numbers	-10, -4, 0, 120, 1034
	Float	Floating point number	-3.08, 7.0, 23.41
	Complex	Complex numbers	3 + 4i, 2 – 2i
Boolean	Bool	Boolean number	0, 1
Sequence	String	Set of characters	'hello'
	List	Set of items separated by comma	[1,2.3,'a']
	Tuple	Set of items separated by comma enclosed in (). It is not editable	(2,'b',3.4)
Missing value	None	Signifies absence of value in a situation	None
Mapping	Dictionary	Holds data items in key-value pairs	{'a':1,'b':2}

Comments

- These are statement ignored by python interpreter during execution.
- It is used add a remark or note in the source code.
- It starts with # (hash sign) in python.

Operators

These are special symbols used to perform specific operation on values. Different types of operator supported in python are given below:

- Arithmetic operator
- Relational operator
- Assignment operator
- Logical operator
- Identity operator
- Membership operator

Arithmetic Operator

Operator	Operation	Description	Example

+	Addition	Used to add numeric values, Used to concatenate strings	`>>> num1=2`
			`>>> num2 = 7`
			`>>> num1+num2`
			`9`
			
			`>>> a='san'`
			`>>> b='techtipnow'`
			`>>> a+b`
			`'santechtipnow'`

-	Subtraction	Used to subtract right side operand from left side operand	`>>> x=13`
			`>>> y=9`
			`>>> x-y`
			`4`

*	Multiplication	Used to multiply values, replicate string when multiplied with number	`>>> x=13`
			`>>> y=8`
			`>>> x*y`
			`104`
			
			`>>> x='techtipnow'`
			`>>> x*2`
			`'techtipnowtechtipnow'`

/	Division	Used to divide left side operand from right side operand	`>>> x=120`
			`>>> y=8`
			`>>> x/y`
			`15.0`

%	Modulus	Divide left side operand from right side operand and returns remainder	`>>> x=93`
			`>>> y=7`
			`>>> x%y`
			`2`

//	Floor division	Divide left side operand from right side operand and return quotient by removing decimal part	`>>> x=87`
			`>>> y=8`
			`>>> x//y`
			`10`

www.techtipnow.in

**	Exponent	Perform exponential (power) calculation on operands	>>> x=3
			>>> y=4
			>>> x**y
			81

Relational Operator			
Operator	**Operation**	**Description**	**Example**

==	Equal to	Compare two operands and return True if both are equal.	>>> x=5
			>>> y=3
			>>> x==y
			False

!=	Not Equal to	Compare two operands and return True if both are not equal.	>>> x=6
			>>> y=4
			>>> x!=y
			True

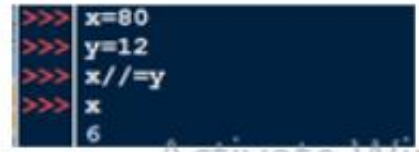
>	Greater than	Compare and return true if left side operand is greater than right side operand	>>> x=6
			>>> y=6
			>>> x>=6
			True

<	Less than	Compare and return true if left side operand is smaller than right side operand	>>> x=43
			>>> y=23
			>>> x<y
			False

>=	Greater than Equal to	Compare and return true if left side operand is greater than or equal to right side operand	>>> x=8
			>>> y=12
			>>> x>=y
			False

<=	Less than equal to	Compare and return true if left side operand is smaller than or equal to right side operand	>>> x=67
			>>> y=32
			>>> x<=y
			False

www.technipnow.in

Assignment Operator		
Operator	**Description**	**Example**
=	Assigns value from right side operand to left side operand	
+=	It adds right side operand to left side operand and assigns the result to the left side operand Ex: x+=y is same as x = x+y	
-=	It subtract right side operand from left side operand and assigns the result to the left side operand Ex: x-=y is same as x = x-y	
*=	It multiply both side operands and assigns the result to the left side operand Ex: x*=y is same as x = x*y	
/=	It divide left side operand from right side operand and assigns the result to the left side operand Ex: x/=y is same as x = x/y	
%=	It divide left side operand from right side operand and assigns the remainder to the left side operand Ex: x%=y is same as x = x%y	
**=	Perform exponential (power) calculation on operands and assigns value to the left side operand Ex: x**=y is same as x = x**y	
//=	It performs floor division using both operands and assigns the result to left side operand. Ex: x//=y is same as x = x//y	

www.techtrbo.in

Activate Win

Logical Operator			
Operator	**Operation**	**Description**	**Example**

and	Logical AND	It returns true only when both the operands is True	`>>> x=12`
			`>>> y=10`
			`>>> bool(x>10 and y==12)`
			`False`

or	Logical OR	It returns true if any one of the operand is True	`>>> x=12`
			`>>> y=10`
			`>>> bool(x>10 or y==12)`
			`True`

not	Logical NOT	It reverses the logical state of operand	`>>> bool(not x==12)` `False`

Identity Operator (Used to identify the type of value in a variable)		
Operator	**Description**	**Example**

is	It returns True when operand's type and data type are same
	It also returns True when both the operands point to same address
		`>>> x=12`
		`>>> y=10`
		`>>> bool(x>10 and y==12)`
		`False`

not is	It returns True when operand's type and data type are not same	`>>> x=12`
		`>>> y=10`
		`>>> bool(x>10 or y==12)`
		`True`

Membership Operator (Used to check existence of a value in given sequence)		
Operator	**Description**	**Example**

in	Returns True if the variable/value is found in the specified sequence and False otherwise	`>>> L=[12,23,34]`
		`>>> 12 in L`
		`True`

not in	Returns True if the variable/value is not found in the specified sequence and False otherwise	`>>> L= [12,23,34]`
		`>>> 12 not in L`
		`False`

www.techtipnow.in

Expression

- An expression is combination of different variables, operators and constant which is always evaluated to a value.
- A value or a standalone variable is also considered as an expression.

Example

56 + (23-13) + 89%8 – 2*3

Evaluation:

```
= 56 + (23 -13) + 89%8 – 2*3      #step1
= 56 + 10 + (89%8) – 2*3          #step2
= 56 + 10 + 1 – (2*3)             #step3
= (56 + 10) + 1 – 6               #step4
= (66 + 1) – 6                    #step5
= 67 – 6                          #step6
= 61
```

Statement

A statement is unit of code that the python interpreter can execute.

Example:

```
var1 = var2                          #assignment statement
x = input ("enter a number")      #input statement
print ("total = ", R)                #output statement
```

How to input values in python?

In python we have input() function for taking user input.

Syntax:

Input([prompt])

How to display output in python?

In python we have print() function to display output.

print([message/value])

Example:

Python program to input and output your name

```
var = input("Enter your name")
print("Name you have entered is ", var)
```

Example:

Addition of two numbers

```
Var1 = int(input("enter no1"))
Var2 = int(input("enter no2"))
Total = Var1 + Var2
Print("Total = ", Total)
```

Type Conversion

Type conversion refers to converting one type of data to another type.

Type conversion can happen in two ways:

- Explicit conversion
- Implicit conversion

Explicit Conversion

- Explicit conversion also refers to type casting.
- In explicit conversion, data type conversion is forced by programmer in program

Syntax:

(new_data_type) = (expression)
Explicit type conversion function

Explicit type conversion function in Python	
Function	**Description**
int(value)	Convert value to an integer
float(value)	Convert value to a floating point number
str(value)	Converts value to string representation
chr(value)	Converts value to a character
unichr(value)	Converts value to a Unicode character

Example:

Program of explicit type conversion from float to int

```
x = 12
y = 5
print(x/y)                  #output – 2.4
print(int(x/y))             #output – 2
```

Program of explicit type conversion from string to int

```
x = input("enter a number")
print(x+2)                          #output – produce error "can only concatenate str to str
x = int(input(Enter a number"))
print(x+2)                          #output – will display addition of value of x and 2
```

Implicit conversion

- Implicit conversion is also known as coercion.
- In implicit conversion data type conversion is done automatically.
- Implicit conversion allows conversion from smaller data type to wider size data type without any loss of information

Example:

Program to show implicit conversion from int to float

```
var1 = 10                   #var1 is integer
var2 = 3.4                  #var2 is float
res  = var1 – var2          #res becomes float automatically after subtraction
print(res)                  #output – 6.6
print(type(res))                    #output – class 'Float'
```

Strings

- String is basically a sequence which is made up of one or more UNICODE characters.
- Character in string can be any letter, digit, whitespace or any other symbol.
- String can be created by enclosing one or more characters in single, double or triple quotes.

Examples:

Accessing characters in a string (INDEX)

- Individual character in a string can be accessed using indexes.
- Indexes are unique numbers assigned to each character in a string to identify them
- Index always begins from 0 and written in square brackets "[]".
- Index must be an zero, positive or negative integer.
- We get IndexError when we give index value out of the range.

Negative Index

- Python allows negative indexing also.
- Negative indices are used when you want to access string in reverse order.
- Starting from the right side, the first character has the index as -1 and the last character (leftmost) has the index –n where n is length of string.

Is string immutable?

- Yes, string is immutable data type. The content of string once assigned cannot be altered than.
- Trying to alter string content may lead an error.

String operations

String supports following operations:

- Concatenation
- Repetition
- Membership
- Slicing

Concatenation

- Concatenation refers to joining two strings.
- Plus ('+') is used as concatenation operator.

Repetition

- Repetition as it name implies repeat the given string.
- Asterisk ('*') is used as repetition operator.

Membership

- Membership operation refers to checking a string or character is part or subpart of an existing string or not.
- Python uses 'in' and 'not in' as membership operator.
- 'in' returns true if the first string or character appears as substring in the second string.
- 'not in' returns true if the first string or character does not appears as substring in the second string.

Slicing

- Extracting a specific part of string or substring is called slicing
- Subset occurred after slicing contains contiguous elements
- Slicing is done using index range like string[start_index : end_index : step_value]
- End index is always excluded in resultant substring.
- Negative index can also be used for slicing.

Traversing a String

- Traversing a string refers to accessing each character of a given string sequentially.
- For or while loop is used for traversing a string

List

- List is built in sequence data type in python.
- Stores multiple values
- List item can be of different data types
- All the items are comma separated and enclosed in square bracket.
- Individual item in a list can be accessed using index which begins from 0.

Examples

```
>>> L = [4,8,9,12,11]
>>> L
[4, 8, 9, 12, 11]
>>> List1 = [12,5.67,'A']
>>> List1
[12, 5.67, 'A']
>>> List2 = ['Eng','Hin','Math','Sci','SST']
>>> List2
['Eng', 'Hin', 'Math', 'Sci', 'SST']
>>>
```

Accessing elements in a List

- Individual item in a list can be accessed using indexes.
- Indexes are unique numbers assigned to each item in a list to identify them
- Index always begins from 0 and written in square brackets "[]".

Example:

```
>>> L = [4,8,9,12,11]
>>> #Display first element of List 'L'
>>> L[0]
4
>>> #Display 3rd element of List 'L'
>>> L[2]
9
>>> #Display Last Element using Negative Index
>>> L[-1]
11
>>> #Display 3rd element using Negative Index
>>> L[-3]
9
```

List is Mutable

- Yes list is mutable.
- Content of the list can be changed after it has been created.

```
>>> L = [4,8,9,12,11]
>>> L[1] = 10
>>> L
[4, 10, 9, 12, 11]
>>>
```

List operations

List supports following operations:

- Concatenation
- Repetition
- Membership
- Slicing

Concatenation

- Concatenation refers to joining two List.
- Plus ('+') is used as concatenation operator.
- Concatenating List with other data type produces TypeErrors.

Example:

```
>>> L1 = [4,7,9]
>>> L2 = [12, 8]
>>> L1+L2
    [4, 7, 9, 12, 8]
>>> L3 = ['a','b']
>>> L4 = ['aa','bb']
>>> L5 = L3 + L4
>>> L5
    ['a', 'b', 'aa', 'bb']
>>> L1 + L3
    [4, 7, 9, 'a', 'b']
```

```
>>> #Concatenating Lists with different types
>>> #causes ValueError
>>> X = 12.6
>>> L2 + X
    Traceback (most recent call last):
      File "<pyshell#44>", line 1, in <module>
        L2 + X
    TypeError: can only concatenate list (not "float") to list
>>>
```

Repetition

- Repetition as it name implies used to replicate a list at specified no of times.
- Asterisk ('*') is used as repetition operator.

```
>>> L = ['a','b']
>>> L*2
    ['a', 'b', 'a', 'b']
>>>
```

Membership

- Membership operation refers to checking an item is exists in the list or not.
- Python uses 'in' and 'not in' as membership operator.
- 'in' returns true if the item specified present in the list.
- 'not in' returns true if the item specified present in the list.

Example:

```
>>> L = ['Eng','Hin','Math','Sci','SST']
>>> 'SST' in L
True
>>> 'Sci' not in L
False
>>>
```

Slicing

- Extracting a subset of items from given list is called slicing
- Subset occurred after slicing contains contiguous items.
- Slicing is done using index range like List[start_index : end_index : step_value]
- End index is always excluded in resultant subset of list.
- Negative index can also be used for slicing.

Examples:

```
>>> L = ['Eng','Hin','Math','Sci','SST', 156, 82,93.04]
>>> L[2:5]
['Math', 'Sci', 'SST']
>>> #Putting End Index out of range
>>> #will display items till end of list
>>> #and List is truncated to the end of list
>>> L[4:50'
...
SyntaxError: unterminated string literal (detected at line
1)
>>> L[4:50]
...
['SST', 156, 82, 93.04]
>>> L[:3]
...
['Eng', 'Hin', 'Math']
>>> L[4:]
...
['SST', 156, 82, 93.04]
>>> #Slicing with given steps size
...
>>> L[1:6:2]
```

```
...
>>> L[1:6:2]
...
    ['Hin', 'Sci', 156]
>>> #slicing with NEGATIVE INDEX
...
>>> L[::-1]
...
    [93.04, 82, 156, 'SST', 'Sci', 'Math', 'Hin', 'Eng']
>>> L[-6:-2]
...
    ['Math', 'Sci', 'SST', 156]
>>>
```

Traversing a List

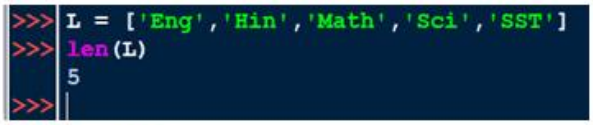
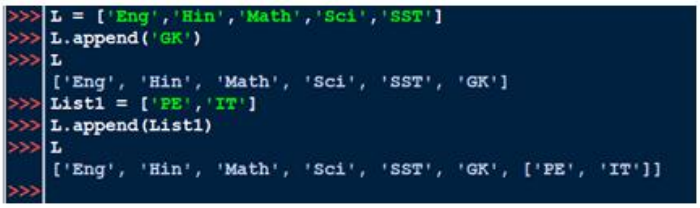
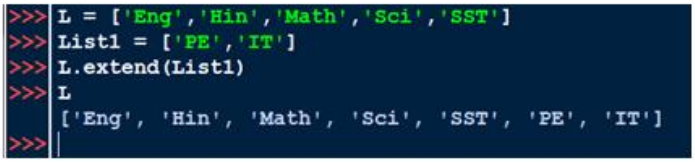
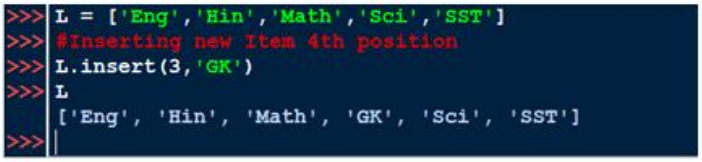
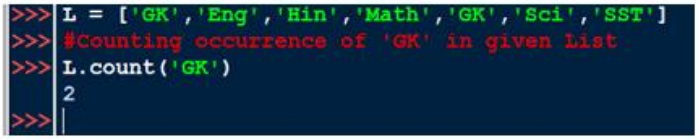
- Traversing a List refers to accessing each item a given list sequentially.
- for or while loop can be used for traversing a list.

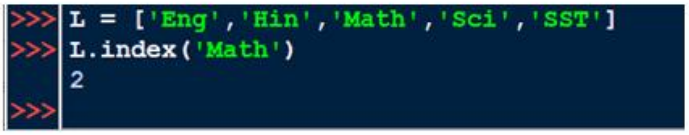
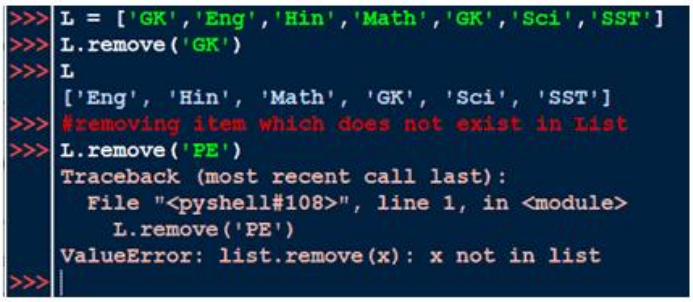
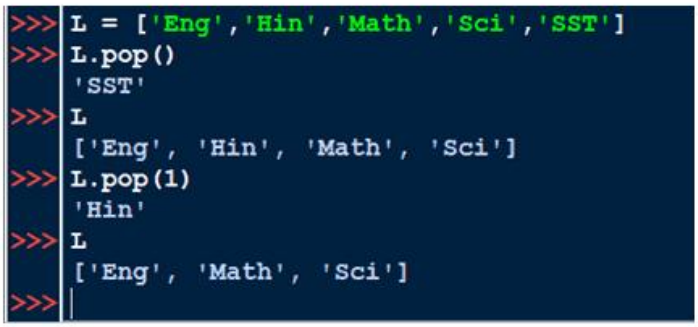
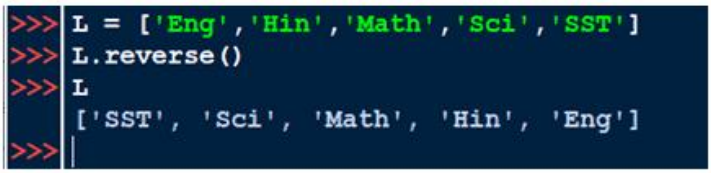
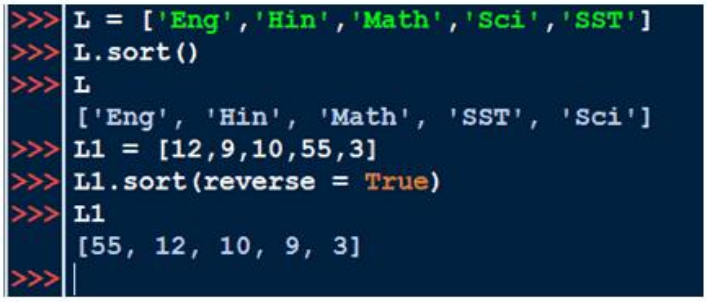
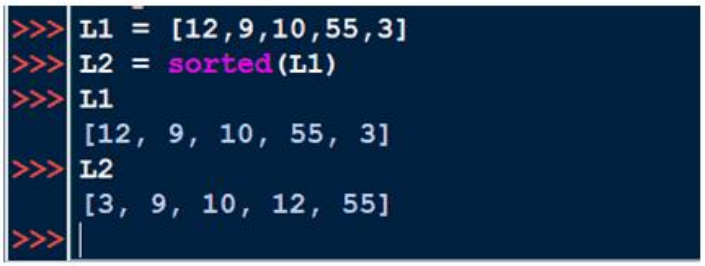
Examples:

```
>>> L = ['Eng','Hin','Math','Sci','SST']
...
>>> #Traversing List using FOR loop
...
>>> for item in L:
...   print(item)
...
...
    Eng
    Hin
    Math
    Sci
    SST
>>> #Traversing List using WHILE loop
...
>>> item = 0
...
>>> while item < len(L):
...     print(L[item])
...     item += 1
```

```
...
...
    Eng
    Hin
    Math
    Sci
    SST
```

List methods | List Functions

Method	Description	Example
len()	Count and returns total no of elements in a list as length	
list()	Create an empty or list with items	
append()	appends an item or list of item as single element at the end of list	
extend()	appends each item of the list individually at the end of list	
insert()	Insert an item at a particular index in the list	
count()	Count and returns a particular item in a given list	

index()	Return index of the first occurrence of the item in the list. If item is not found, ValueError is displayed.	
remove()	Removes the first occurrence of item in the given list. If no item is found, ValueError is displayed.	
pop()	it removes an item from list by passing its index. Bydefault it removes last element of list.	
reverse()	It reverses the order of element in the given list	
sort()	It arranges the items of list in ascending or descending order	
sorted()	It takes a list as parameter and creates a new list consisting of the same elements arranged in sorted order	

```
>>> L = ['Eng','Hin','Math','Sci','SST']
>>> L.index('Math')
2
>>>
```

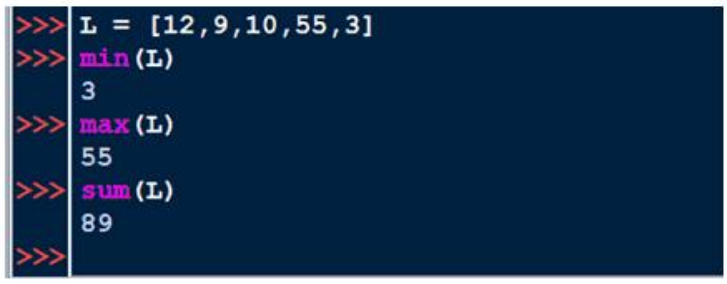
```
>>> L = ['GK','Eng','Hin','Math','GK','Sci','SST']
>>> L.remove('GK')
>>> L
['Eng', 'Hin', 'Math', 'GK', 'Sci', 'SST']
>>> #removing item which does not exist in List
>>> L.remove('PE')
Traceback (most recent call last):
  File "<pyshell#108>", line 1, in <module>
    L.remove('PE')
ValueError: list.remove(x): x not in list
>>>
```

```
>>> L = ['Eng','Hin','Math','Sci','SST']
>>> L.pop()
'SST'
>>> L
['Eng', 'Hin', 'Math', 'Sci']
>>> L.pop(1)
'Hin'
>>> L
['Eng', 'Math', 'Sci']
>>>
```

```
>>> L = ['Eng','Hin','Math','Sci','SST']
>>> L.reverse()
>>> L
['SST', 'Sci', 'Math', 'Hin', 'Eng']
>>>
```

```
>>> L = ['Eng','Hin','Math','Sci','SST']
>>> L.sort()
>>> L
['Eng', 'Hin', 'Math', 'SST', 'Sci']
>>> L1 = [12,9,10,55,3]
>>> L1.sort(reverse = True)
>>> L1
[55, 12, 10, 9, 3]
>>>
```

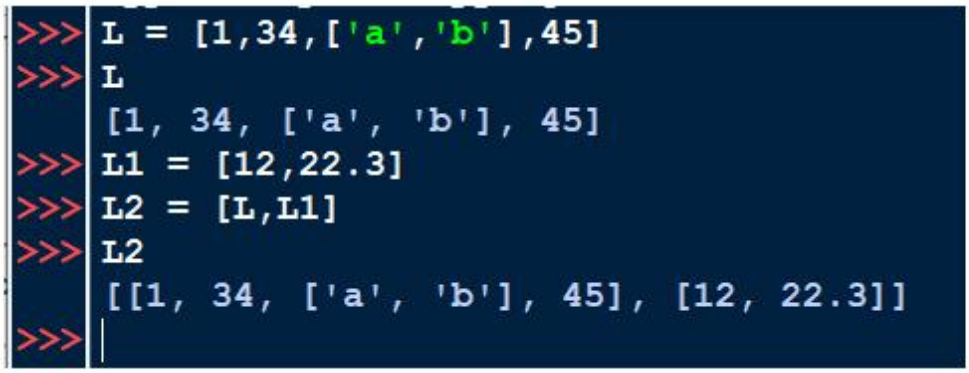
```
>>> L1 = [12,9,10,55,3]
>>> L2 = sorted(L1)
>>> L1
[12, 9, 10, 55, 3]
>>> L2
[3, 9, 10, 12, 55]
>>>
```

min()	Returns smallest element in given list	
max()	Returns largest element in given list	
sum()	Returns sum of all the elements in the given list	

Nested List

One list appears as an element inside another list.

Example:



Tuple

- Tuple is built in sequence data type in python.
- Stores multiple values
- Tuple item can be of different data types
- All the items are comma separated and enclosed in parenthesis '()'.
- Individual item in a Tuple can be accessed using index which begins from 0.
- In case of single item present in tuple, it should also be followed by a comma.
- A sequence without parenthesis is treated as tuple by default.

Examples

```
>>> T = (12,21,33,40)
>>> T
(12, 21, 33, 40)
>>> T1 = (81, 74.22, 'Jan', 'Feb')
>>> T1
(81, 74.22, 'Jan', 'Feb')
>>> T3 = ('Math', 78.9, [44,33])
>>> T3
('Math', 78.9, [44, 33])
>>> T4 = (3,8.2,('a','b'))
>>> T4
(3, 8.2, ('a', 'b'))
>>>
```

Accessing elements in a Tuple

- Individual item in a Tuple can be accessed using indexes.
- Indexes are unique numbers assigned to each item in a Tuple to identify them
- Index always begins from 0 and written in square brackets "[]".

Example:

```
>>> T = (4,8,12,3,1,10,7)
>>> #Display first element
>>> T[0]
4
>>> #Display 4th element
>>> T[3]
3
>>> #Index out of range produces ValueError
>>> T[14]
Traceback (most recent call last):
  File "<pyshell#44>", line 1, in <module>
    T[14]
IndexError: tuple index out of range
>>> #Display last element using Negative Index
>>> T[-1]
7
>>> #Display 4th element using Negative Index
>>> T[-4]
3
>>>
```

Tuple is Immutable

- Yes Tuple is Immutable.
- Content of the Tuple cannot be changed after it has been created.

Example:

```
>>> T = (12,23,34)
>>> T[1] = 50
Traceback (most recent call last):
  File "<pyshell#50>", line 1, in <module>
    T[1] = 50
TypeError: 'tuple' object does not support item ass
ignment
>>>
```

Tuple operations

Tuple supports following operations:

- Concatenation
- Repetition
- Membership
- Slicing

Concatenation

- Concatenation refers to joining two Tuple.
- Plus ('+') is used as concatenation operator.
- Concatenation operator can also be used for extending an existing tuple.

Example:

```
>>> T1 = (53,77)
>>> T2 = ('a','b')
>>> T3 = T1 + T2
>>> T3
(53, 77, 'a', 'b')
>>> #Concatenation operator can also be used for
>>> #extending a tuple by adding new item at end
>>> T3 = T3 + ('jan',)
>>> T3
(53, 77, 'a', 'b', 'jan')
>>>
```

Repetition

- Repetition as it name implies used to repeat elements of a Tuple at specified no of times.
- Asterisk ('*') is used as repetition operator.

```
>>> T = (5,1,3)
>>> T*3
(5, 1, 3, 5, 1, 3, 5, 1, 3)
```

Membership

- Membership operation refers to checking an item exists in Tuple or not.
- Python uses 'in' and 'not in' as membership operator.
- 'in' returns true if the item specified present in the Tuple.
- 'not in' returns true if the item specified present in the Tuple.

Example:

```
>>> T1 = (81, 74.22, 'Jan', 'Feb', 'Mar')
>>> 74.22 in T1
True
>>> 'Apr' not in T1
True
>>>
```

Slicing

- Extracting a subset of items from given Tuple is called slicing
- Subset occurred after slicing contains contiguous items.
- Slicing is done using index range like Tuple[start_index : end_index : step_value]
- End index is always excluded in resultant subset of Tuple.
- Negative index can also be used for slicing.

Examples:

```
>>> T = (22,33,55,11,10,8,6,32,28)
>>> #Display 2nd to 6th element
>>> T[1:6]
(33, 55, 11, 10, 8)
>>> #Display all elements from 3rd index
>>> T[3:]
(11, 10, 8, 6, 32, 28)
>>> #Display first 5 elements
>>> T[:5]
(22, 33, 55, 11, 10)
>>> #Display all elements
>>> T[:]
(22, 33, 55, 11, 10, 8, 6, 32, 28)
>>> #Display every 3rd element
>>> T[::3]
(22, 11, 6)
>>> #Display elements in reverse order
>>> T[::-1]
(28, 32, 6, 8, 10, 11, 55, 33, 22)
>>> #Display elements using negative index
>>> T[-7:-3]
(55, 11, 10, 8)
>>>
```

Traversing a Tuple

- Traversing a Tuple refers to accessing each item a given Tuple sequentially.
- for or while loop can be used for traversing a Tuple.

Examples:

```
>>> T = (23,44,10)
>>> #Traversing using FOR loop
>>> for item in T:
...     print(item)
...
...
23
44
10
>>> #Traversing using WHILE loop
>>> item = 0
>>> while item < len(T):
...     print(T[item])
...     item += 1
...
...
23
44
10
>>>
```

Tuple Methods and Built in Functions

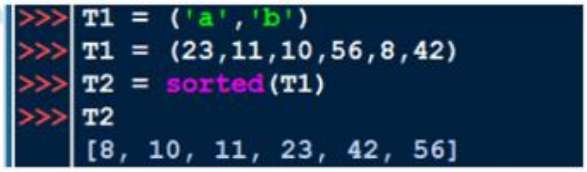
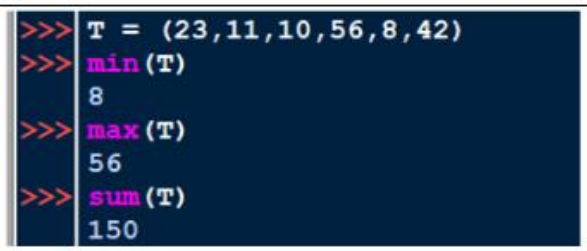
Tuple methods and built in functions		
Method	**Description**	**Example**

len()	Count and returns total no of elements in a Tuple as length	`>>> T = (12,23,44,55)`
		`>>> len(T)`
		`4`

Tuple()	Create an empty or Tuple with items	`>>> T = tuple()`
		`>>> T`
		`()`
		`>>> T1 = tuple('techtipnow')`
		`>>> T1`
		`('t', 'e', 'c', 'h', 't', 'i', 'p', 'n', 'o', 'w')`

count()	Count and returns a particular item in a given Tuple	`>>> T = (12,24,23,56,36,23,11,23)`
		`>>> T.count(23)`
		`3`

index()	Return index of the first occurrence of the item in the Tuple. If item is not found, ValueError is displayed.	`>>> T = ('a','e','i','o','u')`
		`>>> T.index('o')`
		`3`
		`>>> #finding index of item not present in tuple`
		`>>> #will produce ValueError`
		`>>> T.index('x')`
		`Traceback (most recent call last):`
		`  File "<pyshell#29>", line 1, in <module>`
		`  T.index('x')`
		`ValueError: tuple.index(x): x not in tuple`
		`>>>`

sorted()	It takes a tuple as parameter and creates a new tuple consisting of the same elements arranged in sorted order	
min()	Returns smallest element in given Tuple	
max()	Returns largest element in given Tuple	
sum()	Returns sum of all the elements in the given Tuple	

Nested Tuple

One Tuple appears as an element inside another Tuple.

Example:

```
>>> T = (23,44,10,('a','b'))
>>> T
(23, 44, 10, ('a', 'b'))
>>> T1 = (23,22)
>>> T2 = ('m','n')
>>> T3 = (T1,T2)
>>> T3
((23, 22), ('m', 'n'))
>>>
```

Tuple Assignment

Tuple Assignment allows elements of tuple on the left side of assignment operator to be assigned respective values from a tuple on the right side.

```
>>> (a,b) = (10,20)
>>> a
10
>>> b
20
>>> T = (12,'sanjay')
>>> (classs, name)=T
>>> classs
12
>>> name
'sanjay'
>>>
```

Dictionary

- Dictionaries are unordered collection of items falls under mapping.
- Stores data in form of key:value pair called item.
- Key is separated from its value by colon ':' and items are separated by comma.
- All items of dictionaries are enclosed in curly bracket '{}'.
- The keys in the dictionary must be unique and should be of immutable data type.
- The value can be repeated and of any data type.

Creating a Dictionary

```
>>> D = {'a':14,'b':56,'c':23,'d':43}
>>> D
{'a': 14, 'b': 56, 'c': 23, 'd': 43}
>>> D1 = {}
>>> D1
{}
>>> D2 = D
>>> D2
{'a': 14, 'b': 56, 'c': 23, 'd': 43}
>>>
```

Accessing items in a Dictionary

- Items of dictionary are accessed using keys.
- Each key of dictionary serve as index and maps to a value.
- If key is not present in dictionary, then we get KeyError.

Examples

```
>>> #Accessing item which key is not present
>>> D['e']
Traceback (most recent call last):
  File "<pyshell#34>", line 1, in <module>
    D['e']
KeyError: 'e'
>>>
```

Dictionary is Mutable

Yes, Dictionary is Mutable as its content can be changed after it has been created.

Adding a new item in Dictionary

```
>>> D = {'a':12,'b':30}
>>> D['c'] = 55
>>> D
{'a': 12, 'b': 30, 'c': 55}
```

Modifying an existing Item in Dictionary

```
>>> D = {'a':12,'b':30}
>>> D
{'a': 12, 'b': 30}
>>> D['a']=41
>>> D
{'a': 41, 'b': 30}
>>>
```

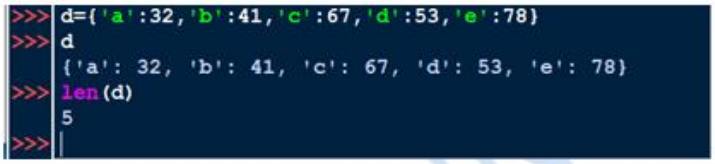
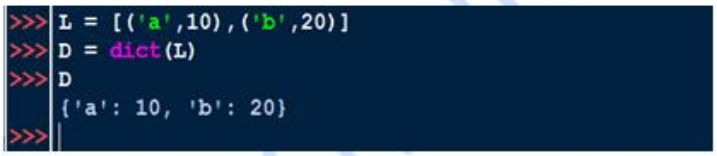
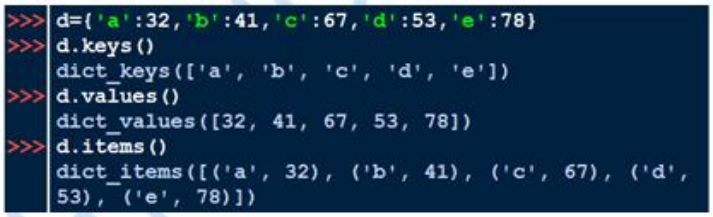
Traversing a dictionary

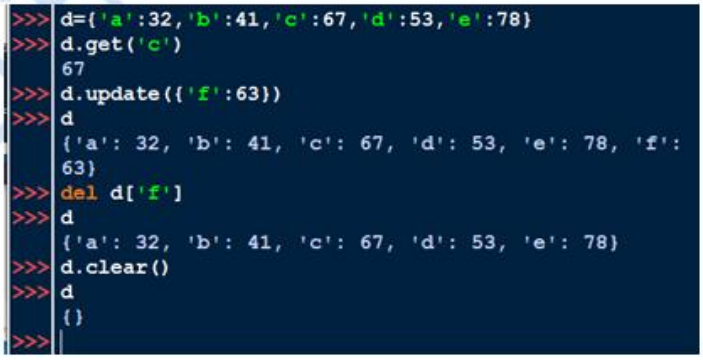
```
>>> D = {'Honda':120,'Suz':200,'Yam':150}
>>> D
{'Honda': 120, 'Suz': 200, 'Yam': 150}
>>> #Method1
>>> for key in D:
...     print(key,':',D[key])
...
...

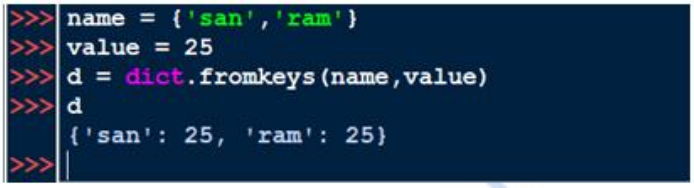
Honda : 120
Suz : 200
Yam : 150
>>> #Method2
>>> for key,value in D.items():
...     print(key,':',value)
...
...

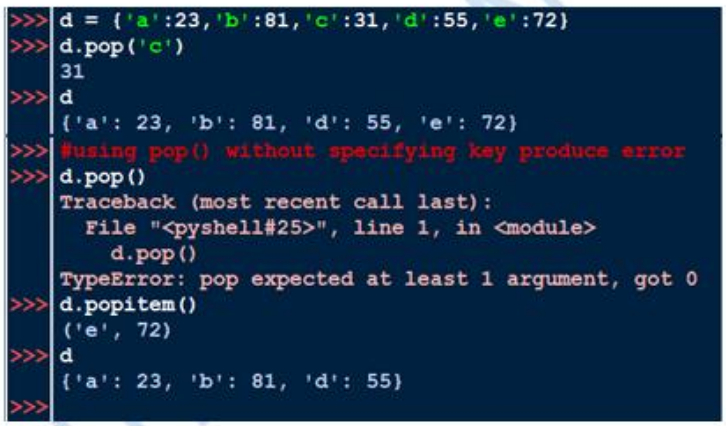
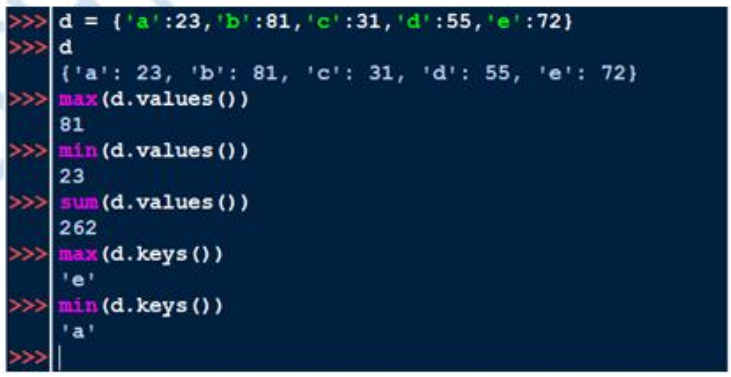
Honda : 120
Suz : 200
Yam : 150
>>>
```

Dictionary methods and built in functions

Dictionary methods and built in functions		
Methods	**Description**	**Example**
len()	Count and return items (key:value pair) of dictionary as length	
dict()	Create a dictionary from sequence of key:value pairs	
keys()	Return all keys in form of List	
values()	Return all values in form of List	
items()	Return list of tuples (key-value) pair	

get()	Display value of the key passed as argument	
update()	Appends new element or modify existing element as key-value pair.	
del()	Removes specific element of dictionary by passing its key as argument	
clear()	Removes all elements from dictionary	

fromkeys()	Dictionary can be created from given sequence of keys and with specified values.	

pop()	Removes the specific element from dictionary by passing key as argument	
popitem()	Removes last element from the dictionary	
max()	Display largest value of dictionary. We can also display largest key.	
min()	Display smallest value of dictionary. We can also find smallest key	
Sum()	Display total of values of dictionary. We can also find sum of keys	